

COMPUTER PROGRAMMING

LAB 12

LAB REPORT

TITLE:

Pointers in C++

OBJECTIVE

The students will learn and apply:

- Computer Memory and Variables
- Pointers
- Pointer Declaration
- Referencing Operator
- Dereferencing Operator

THEORY

COMPUTER MEMORY

- Computer memory is made up of bytes.
Each byte has a unique address.
- Variables are stored in memory locations.
The compiler reserves memory space for every variable

POINTERS IN C++

A pointer is a variable that stores the address of another variable.

Example:

```
int *ptr;
```

- int shows pointer type.
- shows pointer variable.
- ptr is pointer name.

REFERENCING OPERATOR (&)

- The & operator gives the address of a variable.

Example:

```
ptr = &num;
```

DEREFERENCING OPERATOR (*)

- The * operator accesses the value stored at the address.

Example:

```
*ptr = 10;
```

LAB TASK 1

POINTER DECLARATION AND INITIALIZATION

PROBLEM STATEMENT

- Declares integer variable num
- Declares pointer ptr
- Stores address of num in ptr
- Displays value of num
- Displays value pointed by ptr

```
num = 25 | num |  
| 25 |  
+-----+  
Address  
100  
  
&  
|  
|  
  
+-----+  
ptr -----> | 100 |  
+-----+
```

ALGORITHM

1. Start
2. Declare integer variable
3. Declare pointer variable
4. Store address using &
5. Display variable value
6. Display dereferenced value
7. End

PROGRAM

```

#include <iostream>
using namespace std;

int main()
{
    // Declaring integer variable
    int num = 25;

    // Declaring pointer
    int *ptr;

    // Storing address of num
    ptr = &num;

    // Displaying values
    cout << "Value of num = " << num << endl;
    cout << "Value pointed by ptr = " << *ptr << endl;

    return 0;
}

```

OUTPUT

Value of num = 25
Value pointed by ptr = 25

EXPLANATION

- ptr stores address of num.
- *ptr accesses value stored at that address.
- Both outputs are same because pointer points to num.

LAB TASK 2

POINTER ARITHMETIC

PROBLEM STATEMENT

Write a program that:

- Declares integer array numbers
- Uses pointer arithmetic
- Displays array elements using pointer

Array: numbers[] = {10, 20, 30, 40, 50}

```

+----+----+----+----+----+
Index | 0 | 1 | 2 | 3 | 4 |
+----+----+----+----+----+
Value | 10 | 20 | 30 | 40 | 50 |
+----+----+----+----+----+
Address 100 104 108 112 116

```

```

ptr
|
v
+----+
| 10 |
+----+

```

```

(ptr + 1) ---> 20
(ptr + 2) ---> 30
(ptr + 3) ---> 40
(ptr + 4) ---> 50

```

ALGORITHM

1. Start
2. Declare array
3. Declare pointer
4. Point pointer to first element
5. Use pointer arithmetic
6. Display elements
7. End

PROGRAM

```

#include <iostream>
using namespace std;

int main()
{
    // Declaring array
    int numbers[] = {10, 20, 30, 40, 50};

    // Pointer declaration
    int *ptr;

    // Pointer points to first element
    ptr = numbers;

    // Displaying array using pointer arithmetic
    cout << "Array elements are:" << endl;

    for(int i = 0; i < 5; i++)
    {
        cout << *(ptr + i) << endl;
    }

    return 0;
}

```

OUTPUT

Array elements are:

```

10
20
30
40
50

```

EXPLANATION

- ptr stores address of first array element.
- (ptr + i) moves pointer to next element.
- *(ptr + i) accesses value at that location.

IMPORTANT FORMULAS

Address Representation

`ptr=&numptr=\&numptr=&num`

Dereferencing Representation

`*ptr=value*ptr=value*ptr=value`

RESULT

- The programs were executed successfully and the concepts of pointers, referencing, dereferencing, and pointer arithmetic were understood.

CONCLUSION

In this lab:

- We learned computer memory concepts.
- We studied pointers and addresses.
- We used referencing and dereferencing operators.
- We performed pointer arithmetic with arrays.

Pointers help in:

- Efficient memory handling
- Dynamic programming
- Array manipulation